
Project for SE-2:

Book Store System

Team Members:

Mohamed Walid	20230516
Mohamed Anani	20230494
Mohamed Eid	20230495
Mohamed Mostafa	20230507
Sayed Saber	20230270
Ahmed Mohamed	20220038
Naglaa Mohamed	20220530

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Book Store System. The system is a full-stack e-commerce platform for purchasing, managing, and browsing books online. It consists of a Spring Boot backend REST API and a dedicated Notification microservice.

This document is intended for:

- Development and engineering teams responsible for building the system
- Project managers overseeing the development lifecycle
- Quality assurance engineers performing validation and testing
- Stakeholders and product owners reviewing system capabilities

The SRS serves as the authoritative reference for all parties involved in the design, development, testing, and deployment of the Book Store System.

1.2 Scope

The Book Store System is an online bookstore application named BookStore, developed under the package namespace `com.codespring.bookstore`. The system provides:

- A public-facing REST API (running on port 8080) enabling users to browse, search, and purchase books
- An administrative interface for managing the catalog, users, orders, and viewing business statistics
- A dedicated Notification Microservice (running on port 8081) responsible for sending transactional email notifications such as OTP codes for password resets
- JWT-based stateless authentication and role-based access control (USER and ADMIN roles)
- File upload capabilities for book cover images, category images, and user profile pictures
- A shopping cart, favourites list, and order management system

The system is containerized using Docker Compose and consists of four containers: the main MySQL database, the Book Store Spring Boot application, a dedicated notification MySQL database, and the Notification Spring Boot microservice.

Out of scope for this version:

- A frontend web or mobile client application (the backend API is designed for integration with any frontend)
- Payment gateway integration (orders are placed but payment processing is external)
- Real-time notifications (e.g., WebSockets or push notifications)

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
API	Application Programming Interface — a set of HTTP endpoints exposed by the backend
REST	Representational State Transfer — architectural style for the web API
JWT	JSON Web Token — a compact, digitally signed token used for stateless authentication
OTP	One-Time Password — a 6-digit code sent via email for password reset verification
RBAC	Role-Based Access Control — access is granted based on assigned roles (USER, ADMIN)
ADMIN	A user role with full system management privileges
USER	A standard registered user role with shopping and browsing privileges
DTO	Data Transfer Object — a plain object used to transfer data between layers
JPA	Java Persistence API — the ORM standard used for database access
AOP	Aspect-Oriented Programming — used for cross-cutting concerns (logging, auditing, performance)
BCrypt	A password hashing algorithm used to store passwords securely
Docker	A containerization platform used to package and run the application services
Docker Compose	A tool for defining and running multi-container Docker applications
MySQL	The relational database management system used for data persistence
SMTP	Simple Mail Transfer Protocol — used to send email via Gmail
CORS	Cross-Origin Resource Sharing — HTTP headers controlling browser access from different origins

Term / Acronym	Definition
DXA	Device-independent units used in document formatting (not relevant to code)
HTTP	HyperText Transfer Protocol — the communication protocol for the REST API
CRUD	Create, Read, Update, Delete — the four basic data operations

2. Overall Description

2.1 System Perspective

The Book Store System operates as a distributed, microservices-based backend platform. The system is composed of two independent Spring Boot applications coordinated through Docker Compose, both sharing a common deployment environment but maintaining separate concerns and separate databases.

2.1.1 Architecture Overview

The system follows a layered architecture within the main application:

Layer	Components	Responsibility
Presentation	Controllers (REST)	Receives HTTP requests, validates input, returns HTTP responses
Service	Service classes	Implements business logic, orchestrates operations
Persistence	Repositories (Spring Data JPA)	Handles database queries and data access
Security	JwtAuthenticationFilter, SecurityConfig	Authenticates requests and enforces RBAC
Cross-Cutting	Aspects (AOP)	Handles logging, auditing, performance monitoring, login tracking
Data Transfer	DTOs and MapStruct Mappers	Converts between entities and API response/request models

2.1.2 Microservices Structure

The deployment consists of four Docker containers:

Container Name	Technology	Port	Role
bookstore-app	Spring Boot 3	8080	Main Book Store API
bookstore-mysql	MySQL 8.0	3306	Database for main application
notification-app	Spring Boot 3	8081	Email notification microservice
notification-mysql	MySQL 8.0	3307 (host)	Database for notification logs

2.1.3 Key Technical Stack

- Backend Framework: Spring Boot 3 with Spring MVC, Spring Data JPA, Spring Security
- Authentication: JWT (JJWT library) with stateless sessions and token blacklisting
- Database: MySQL 8.0 with Hibernate ORM (DDL auto-update mode)
- API Documentation: SpringDoc OpenAPI (Swagger UI available at /swagger-ui)
- Security: Spring Security with BCrypt password encoding
- Cross-Cutting Concerns: Spring AOP for logging, auditing, performance monitoring
- File Storage: Local filesystem with UUID-prefixed filenames under /uploads
- Email: JavaMail via Gmail SMTP with STARTTLS
- Containerization: Docker with Docker Compose orchestration
- Build Tool: Apache Maven
- Mapping: MapStruct for entity-to-DTO conversions
- Utilities: Project Lombok for boilerplate reduction

2.2 User Classes and Characteristics

The system defines two primary user roles with distinct permissions and interaction patterns:

Characteristic		Registered User (USER)	Administrator (ADMIN)
Authentication		JWT Bearer token required	JWT Bearer token required
Account Creation		Already registered	Created by existing admin
Browse Books & Categories		Yes (read-only)	Yes (full access)
Search Books		Yes	Yes
Shopping Cart		Full cart management	No
Place Orders		Yes	No
View Own Orders		Yes	Yes (all orders)
Favourites List		Yes	No
Manage Own Profile		Yes	Yes
Manage Books Catalog		No	Full CRUD
Manage Categories		No	Full CRUD
Manage Users		No	Full CRUD
Update Order Status		No	Yes
View Statistics Dashboard		No	Yes

Characteristic		Registered User (USER)	Administrator (ADMIN)
View Login Statistics		No	Yes
Create Admin Accounts		No	Yes

2.2.1 Guest / Anonymous Users

These are unauthenticated visitors who can browse the catalog without creating an account. They can view all books and categories, search for books, and access the registration and login endpoints. No purchase or personalization features are available without authentication.

2.2.2 Registered Users (Role: USER)

Registered users have a personal account with a profile, shopping cart, favourites list, and order history. They can browse, add items to their cart, place orders, and manage their own profile and password. They cannot access administrative functions or other users' data.

2.2.3 Administrators (Role: ADMIN)

Administrators have elevated privileges to manage the entire system. They can create and modify book listings and categories (including uploading images), view and manage all user accounts, update order statuses, and access the statistics dashboard. Admin accounts can only be created by an existing administrator via the dedicated endpoint.

2.3 Design and Implementation Constraints

2.3.1 Technology Constraints

- The backend must be implemented in Java using the Spring Boot 3 framework.
- All data must be persisted in MySQL 8.0. No other database engine is supported in the current configuration.
- The system uses Hibernate with DDL auto-update, meaning schema changes are applied automatically on startup without migration scripts (Flyway and Liquibase are explicitly disabled).
- File uploads are limited to 5 MB per file and 5 MB per request, as configured in Spring Boot multipart settings.
- Uploaded files are stored on the local filesystem. In the Docker environment, they are stored in a shared volume mounted at /app/uploads.
- JWT tokens expire after 24 hours. The secret key must be at least 256 bits long.
- Email sending is exclusively handled by the Notification microservice using Gmail SMTP on port 587 with STARTTLS.

2.3.2 Architectural Constraints

-
- The system must operate as stateless REST API services. No server-side HTTP sessions are maintained (SessionCreationPolicy.STATELESS).
 - CSRF protection is disabled, as the API is stateless and token-based.
 - Inter-service communication between the main application and the notification service uses synchronous HTTP REST calls (RestTemplate). The notification service is expected to be reachable at <http://notification-app:8081> within the Docker network.
 - The main application depends on the MySQL container being healthy before starting (Docker Compose healthcheck dependency).

2.3.3 Security Constraints

- All passwords must be stored hashed using BCrypt. Plain-text passwords must never be persisted.
- All non-public endpoints require a valid JWT Bearer token in the Authorization header.
- User data (profile, orders, cart) can only be accessed by the owner or an admin.
- Admin-only operations are protected at both the HTTP security layer (SecurityConfig) and the method level (@PreAuthorize annotations).
- OTPs for password reset expire after 15 minutes and are invalidated after successful use.

2.3.4 Operational Constraints

- The system is deployed using Docker Compose with four containers. All containers must be running for full functionality.
- The application exposes its API at the root context path (no /api prefix by default). The notification service API is prefixed with /api/notifications.
- Swagger UI documentation is publicly accessible at /swagger-ui for development and testing purposes.

3. Functional Requirements

3.1 User Management and Authentication

FR-1.1: User Registration

The system shall allow any visitor to register a new user account by providing the required details. The system shall validate input fields and return a 201 Created response with the created user profile. Duplicate email addresses shall be rejected.

- Endpoint: POST /users/register
- Access: Public
- Input: RegisterUserDto (name, email, password, and other required fields)
- Output: UserDto with the newly created user details

FR-1.2: User Login

The system shall authenticate a user with their email and password. Upon successful authentication, the system shall return a JWT token that must be used for all subsequent authenticated requests. Login events shall be tracked per day.

- Endpoint: POST /users/login
- Access: Public
- Input: LoginRequest (email, password)
- Output: LoginResponse (JWT token and user details)

FR-1.3: User Logout

The system shall allow an authenticated user to invalidate their current JWT token. The token shall be added to a blacklist and rejected for all future requests, even if it has not yet expired.

- Endpoint: POST /users/logout
- Access: Authenticated (USER or ADMIN)
- Input: Authorization header containing Bearer token
- Output: Success message confirming logout

FR-1.4: Forgot Password (OTP Generation)

The system shall allow a user who has forgotten their password to request an OTP reset code. The system shall generate a 6-digit OTP, store it with a 15-minute expiry on the user record, and delegate email sending to the Notification microservice.

- Endpoint: POST /users/forgot-password
- Access: Public
- Input: ForgotPasswordRequest (email)

-
- Output: Confirmation message that OTP was dispatched

FR-1.5: Reset Password

The system shall allow a user to set a new password by providing their email, the OTP received by email, and the desired new password. The system shall verify the OTP is correct and has not expired. Upon success, the new password shall be BCrypt-hashed and saved, and the OTP invalidated.

- Endpoint: POST /users/reset-password
- Access: Public
- Input: ResetPasswordRequest (email, otp, newPassword)

FR-1.6: Change Password

The system shall allow an authenticated user to change their password by providing their current password and a new one.

- Endpoint: POST /users/{id}/change-password
- Access: Authenticated
- Input: ChangePasswordRequest

FR-1.7: Get All Users

The system shall allow administrators to retrieve a list of all registered users.

- Endpoint: GET /users
- Access: ADMIN only
- Output: List of UserDto

FR-1.8: Get User by ID

The system shall allow a user to retrieve their own profile, and administrators to retrieve any user profile.

- Endpoint: GET /users/{id}
- Access: Owner or ADMIN (@securityChecks.isOwner)

FR-1.9: Edit User Profile

The system shall allow a user to update their own profile information, optionally including a profile picture upload.

- Endpoint: PUT /users/{id} (multipart/form-data)
- Access: Owner only
- Input: UserDto fields and optional image file

FR-1.10: Upload User Profile Image

The system shall allow uploading or replacing a user's profile picture independently.

- Endpoint: POST /users/{userId}/upload-image

-
- Access: Authenticated

FR-1.11: Delete User

The system shall allow administrators to permanently delete a user account.

- Endpoint: DELETE /users/{id}
- Access: ADMIN only

FR-1.12: Create Admin Account

The system shall allow an existing administrator to create a new administrator account with the ADMIN role.

- Endpoint: POST /users/create-admin
- Access: ADMIN only
- Input: CreateAdminRequest

3.2 Book Management

FR-2.1: Get All Books

The system shall return a complete list of all books in the catalog.

- Endpoint: GET /books
- Access: Public
- Output: List of BookDto

FR-2.2: Get Book by ID

The system shall return the details of a specific book identified by its unique ID.

- Endpoint: GET /books/{id}
- Access: Public

FR-2.3: Search Books by Title

The system shall support searching for books by a partial or full title string.

- Endpoint: GET /books/search?title={query}
- Access: Public

FR-2.4: Get Books by Category

The system shall allow filtering the book catalog by category name.

- Endpoint: GET /books/category/{categoryName}
- Access: Public

FR-2.5: Create Book

The system shall allow administrators to add a new book to the catalog, including an optional cover image upload.

- Endpoint: POST /books (multipart/form-data)
- Access: ADMIN only
- Input: BookDto fields and optional image file

FR-2.6: Update Book

The system shall allow updating the details and cover image of an existing book.

- Endpoint: PUT /books/{id} (multipart/form-data)
- Access: Authenticated (ADMIN recommended)

FR-2.7: Delete Book

The system shall allow administrators to permanently remove a book from the catalog.

- Endpoint: DELETE /books/{id}
- Access: ADMIN only

3.3 Category Management

FR-3.1: Get All Categories

The system shall return a list of all book categories.

- Endpoint: GET /categories
- Access: Public

FR-3.2: Get Category by ID

The system shall return the details of a specific category.

- Endpoint: GET /categories/{id}
- Access: Public

FR-3.3: Create Category

The system shall allow administrators to create a new book category with an optional category image.

- Endpoint: POST /categories (multipart/form-data)
- Access: ADMIN only

FR-3.4: Update Category

The system shall allow administrators to update the name or image of an existing category.

- Endpoint: PUT /categories/{id}

-
- Access: ADMIN only

FR-3.5: Delete Category

The system shall allow administrators to delete a category.

- Endpoint: DELETE /categories/{id}
- Access: ADMIN only

3.4 Shopping Cart

FR-4.1: Get Cart

The system shall return the current shopping cart contents for a given user, including all cart items and their quantities.

- Endpoint: GET /cart/{userId}
- Access: USER only

FR-4.2: Add Item to Cart

The system shall allow a user to add a book to their cart with a specified quantity. If the book is already in the cart, the quantity shall be updated accordingly.

- Endpoint: POST /cart/{userId}/items
- Access: USER only
- Input: AddToCartRequest (bookId, quantity)

FR-4.3: Update Cart Item Quantity

The system shall allow a user to change the quantity of a specific book in their cart. Quantities of zero or less shall be rejected.

- Endpoint: PUT /cart/{userId}/items/{bookId}
- Access: USER only
- Input: UpdateCartItemRequest (quantity)

FR-4.4: Remove Item from Cart

The system shall allow a user to remove a specific book from their cart.

- Endpoint: DELETE /cart/{userId}/items/{bookId}
- Access: USER only

FR-4.5: Clear Cart

The system shall allow a user to remove all items from their cart at once.

- Endpoint: DELETE /cart/{userId}/clear
- Access: USER only

3.5 Order Management

FR-5.1: Place Order

The system shall allow a user to convert the contents of their shopping cart into a placed order. The cart shall be cleared upon successful order placement.

- Endpoint: POST /orders/place/{userId}
- Access: USER only
- Output: OrderDto with order details and items

FR-5.2: Get Orders by User

The system shall allow a user to retrieve their full order history.

- Endpoint: GET /orders/user/{userId}
- Access: USER only

FR-5.3: Get All Orders

The system shall allow administrators to retrieve all orders across all users.

- Endpoint: GET /orders
- Access: ADMIN only

FR-5.4: Get Order by ID

The system shall allow administrators to retrieve the details of a specific order.

- Endpoint: GET /orders/{id}
- Access: ADMIN only

FR-5.5: Update Order Status

The system shall allow administrators to update the status of an order (e.g., from PENDING to SHIPPED or DELIVERED).

- Endpoint: PATCH /orders/{id}/status
- Access: ADMIN only
- Input: UpdateOrderStatusRequest (status)

3.6 Favourites

FR-6.1: Get User Favourites

The system shall allow a user to retrieve their list of favourite books.

- Endpoint: GET /favourites/user/{userId}

-
- Access: USER only

FR-6.2: Add Favourite

The system shall allow a user to mark a book as a favourite.

- Endpoint: POST /favourites/{userId}/books/{bookId}
- Access: USER only

FR-6.3: Remove Favourite

The system shall allow a user to remove a book from their favourites list.

- Endpoint: DELETE /favourites/{userId}/books/{bookId}
- Access: USER only

3.7 Statistics and Analytics (Admin Dashboard)

FR-7.1: Get Dashboard Statistics

The system shall provide administrators with a summary statistics dashboard including totals for users, books, orders, and revenue.

- Endpoint: GET /admin/statistics
- Access: ADMIN only
- Output: StatisticsDto

FR-7.2: Get Orders Chart Data

The system shall provide administrators with order volume data for the past 7 days, suitable for rendering charts.

- Endpoint: GET /admin/statistics/chart/orders
- Access: ADMIN only
- Output: List of ChartItemDto (date, count)

FR-7.3: Get Login Statistics (All Days)

The system shall provide administrators with a map of all login activity per day, tracking how many users logged in on each date.

- Endpoint: GET /users/stats/logins
- Access: ADMIN only
- Output: Map<String, Integer> (date string to login count)

FR-7.4: Get Login Statistics (Specific Day)

The system shall allow administrators to query the login count for a specific calendar date.

- Endpoint: GET /users/stats/logins/{date}

-
- Access: ADMIN only
 - Output: Map with date and count fields

3.8 Notification Microservice

FR-8.1: Send Email Notification

The Notification microservice shall expose an endpoint to accept email sending requests from the main application. It shall send the email via Gmail SMTP and log the notification in its own dedicated database.

- Endpoint: POST /api/notifications/send
- Access: Internal service call (not publicly exposed)
- Input: NotificationRequest (toEmail, subject, body)
- Output: Confirmation of dispatch

FR-8.2: Notification Logging

All email notifications sent through the microservice shall be persisted as NotificationLog records in the notification database for audit and troubleshooting purposes.

4. Non-Functional Requirements

4.1 Security

ID	Requirement	Details
NFR-S-1	Authentication	All protected endpoints must require a valid JWT Bearer token. Expired or blacklisted tokens must be rejected.
NFR-S-2	Authorization	Role-based access must be enforced at both the HTTP routing layer (SecurityConfig) and method level (@PreAuthorize). Users must not access other users' resources.
NFR-S-3	Password Security	All passwords must be stored using BCrypt hashing. Plain-text passwords must never appear in the database or API responses.
NFR-S-4	Token Blacklisting	Logged-out JWT tokens must be blacklisted and rejected for their remaining lifetime, preventing reuse after logout.
NFR-S-5	OTP Expiry	Password reset OTPs must expire after 15 minutes. Expired OTPs must be rejected and the user must request a new one.
NFR-S-6	Input Validation	All user input must be validated using Jakarta Bean Validation annotations. Invalid input must return a 400 Bad Request with an informative error response.
NFR-S-7	CORS	The system must configure appropriate CORS settings (WebConfig) to allow legitimate frontend clients to communicate with the API.

4.2 Performance

ID	Requirement	Details
NFR-P-1	Response Time	API endpoints for reading data (books, categories, cart) should respond within 500 milliseconds under normal load conditions.
NFR-P-2	File Upload Size	The system must enforce a maximum upload size of 5 MB per file and per request. Larger uploads must be rejected.
NFR-P-3	Performance Monitoring	The PerformanceAspect AOP component must log execution time for all service methods. Methods exceeding defined thresholds should be flagged in logs.
NFR-P-4	Database Indexing	Frequently queried fields (e.g., email in User, title in Book, category in Book) should be indexed to support fast lookups.

4.3 Reliability and Availability

ID	Requirement	Details
NFR-R-1	Container Restart Policy	All Docker containers must be configured with restart: always to automatically recover from unexpected crashes.
NFR-R-2	Database Healthcheck	The main application and notification service containers must not start until their respective MySQL databases pass a health check, preventing startup failures due to DB unavailability.
NFR-R-3	Data Persistence	All database data must be persisted using Docker named volumes (mysql-data, notification-mysql-data) to survive container restarts.
NFR-R-4	Error Handling	The GlobalExceptionHandler must intercept all unhandled exceptions and return structured error responses. Unhandled exceptions must not expose stack traces to clients.
NFR-R-5	Notification Resilience	Failures in the notification microservice (e.g., SMTP errors) must be communicated clearly to the main application. The main application should not silently ignore notification failures.

4.4 Maintainability

ID	Requirement	Details
NFR-M-1	Code Structure	The codebase must follow a clear layered package structure: controllers, services, repositories, entities, dtos, mappers, security, aspects, config.
NFR-M-2	Logging	The LoggingAspect and AuditAspect must log all significant operations, including method entry/exit, parameters, and outcomes, to support debugging and auditing.
NFR-M-3	Audit Trail	The AuditAspect must record state-changing operations (create, update, delete) for audit purposes.
NFR-M-4	API Documentation	The system must expose a Swagger UI (via SpringDoc OpenAPI) at /swagger-ui for all endpoints, enabling easy exploration and testing by developers.
NFR-M-5	DTO Separation	All API request and response payloads must use DTOs. JPA entities must not be exposed directly in API responses.
NFR-M-6	Configuration Externalization	All environment-specific configuration (database URLs, credentials, JWT secret, mail settings) must be externalized via environment variables in the Docker Compose configuration.

4.5 Portability and Deployability

ID	Requirement	Details
NFR-D-1	Containerization	The system must be fully deployable using a single docker-compose up command. All service dependencies must be declared in docker-compose.yml.
NFR-D-2	Environment Independence	The application must not hardcode environment-specific values (database URLs, credentials) in source code or application.properties for production deployments; these must come from Docker environment variables.
NFR-D-3	Build Reproducibility	The application must be buildable using Maven (mvn package) without requiring external resources beyond declared pom.xml dependencies.

4.6 Usability (API Usability)

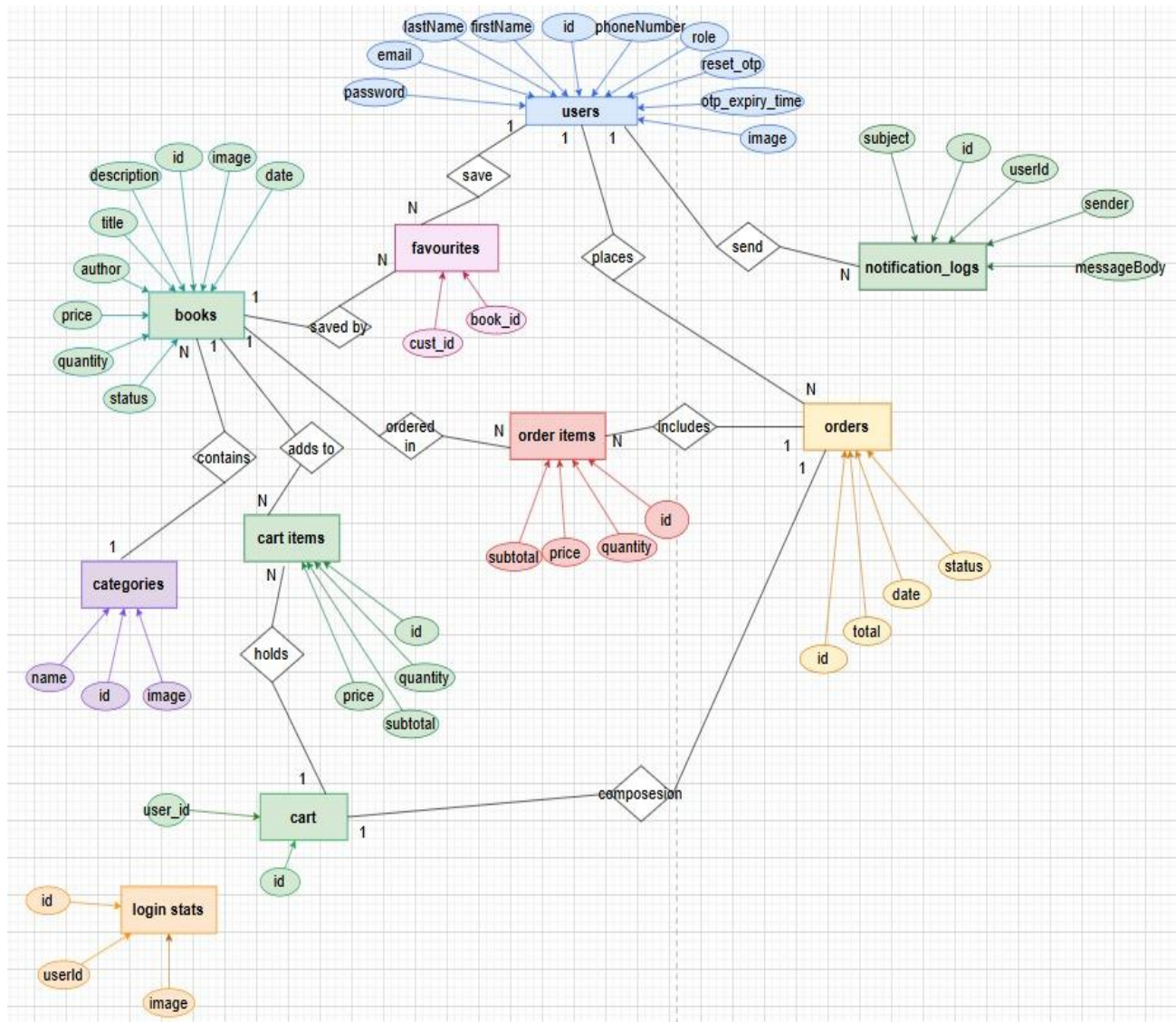
ID	Requirement	Details
NFR-U-1	Consistent Response Format	All API responses must follow consistent HTTP status codes: 200 OK for successful reads, 201 Created for resource creation, 204 No Content for deletions, 400 Bad Request for validation errors, 401/403 for authentication/authorization failures, 404 for not found.
NFR-U-2	RESTful Design	All endpoints must follow REST conventions with meaningful resource-based URL paths and appropriate HTTP methods (GET, POST, PUT, PATCH, DELETE).
NFR-U-3	Informative Errors	Error responses from the GlobalExceptionHandler must include a descriptive message that helps clients understand what went wrong, without exposing internal implementation details.

4.7 Scalability

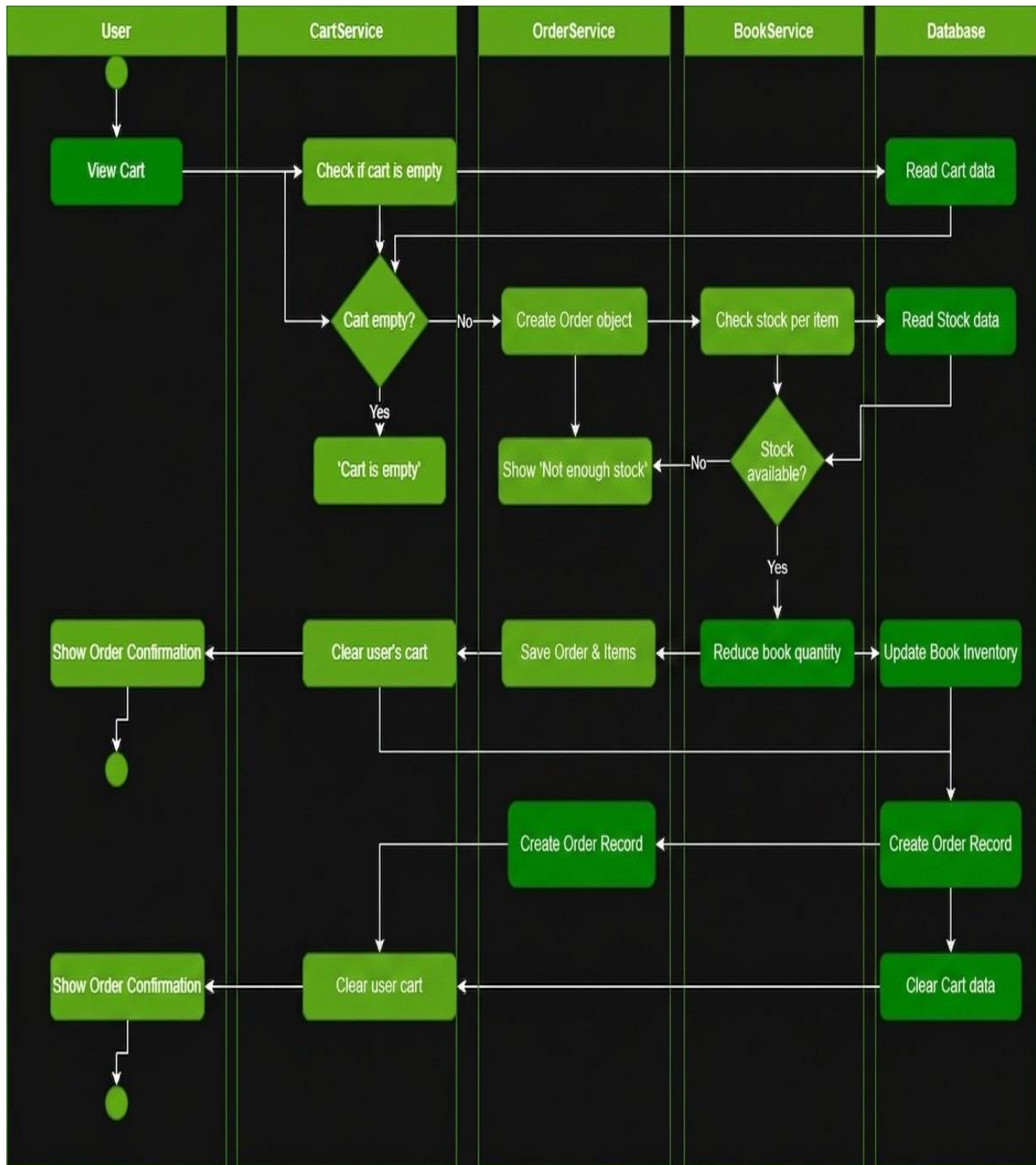
ID	Requirement	Details
NFR-SC-1	Stateless Design	The system must remain stateless (no server-side sessions) to enable horizontal scaling. Multiple instances of the main application must be operable behind a load balancer without session sharing.
NFR-SC-2	Microservice Independence	The notification service must operate independently and be scalable separately from the main application. Its only integration point is the /api/notifications/send HTTP endpoint.

Diagrams:

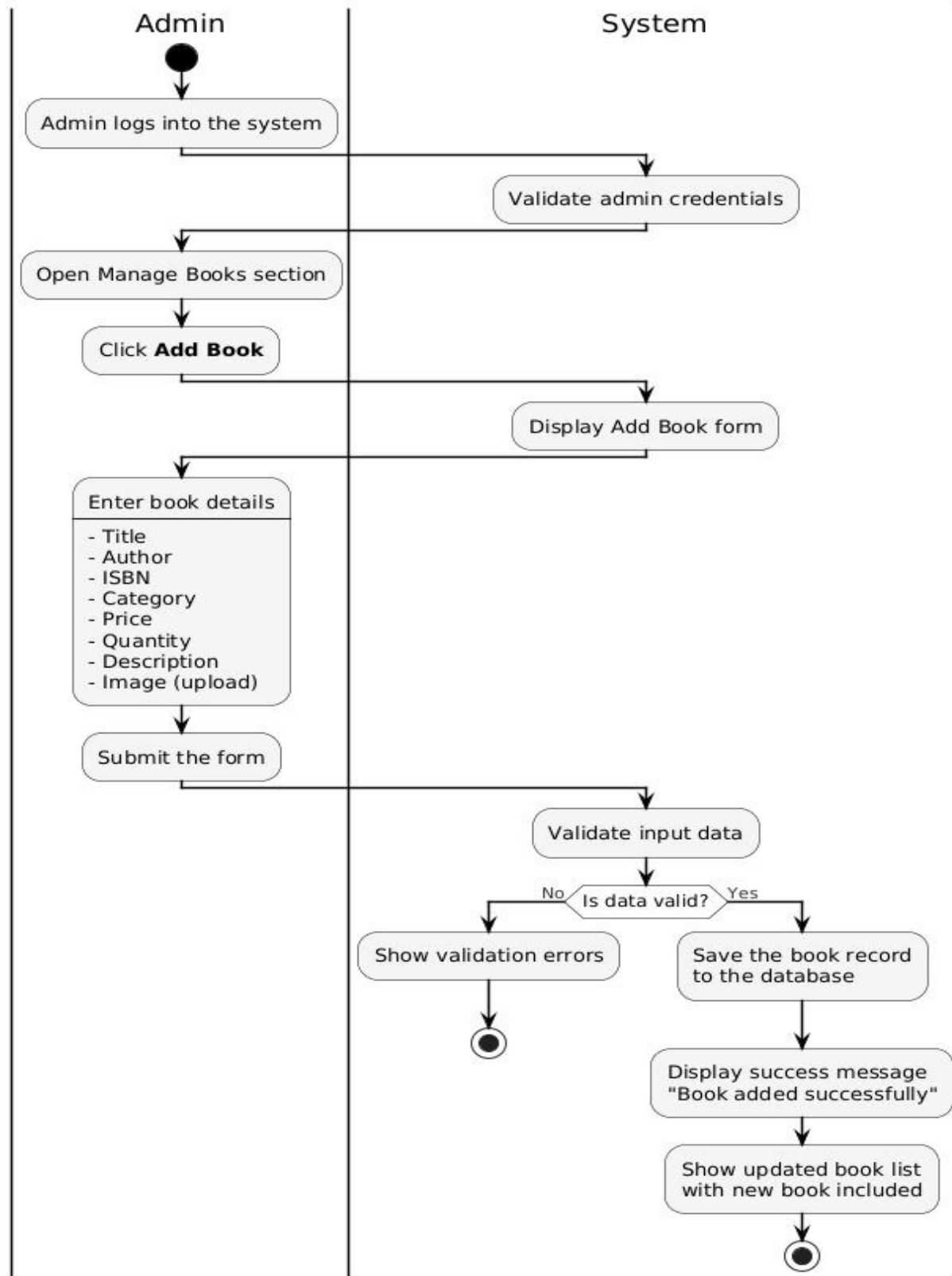
ERD Diagram:



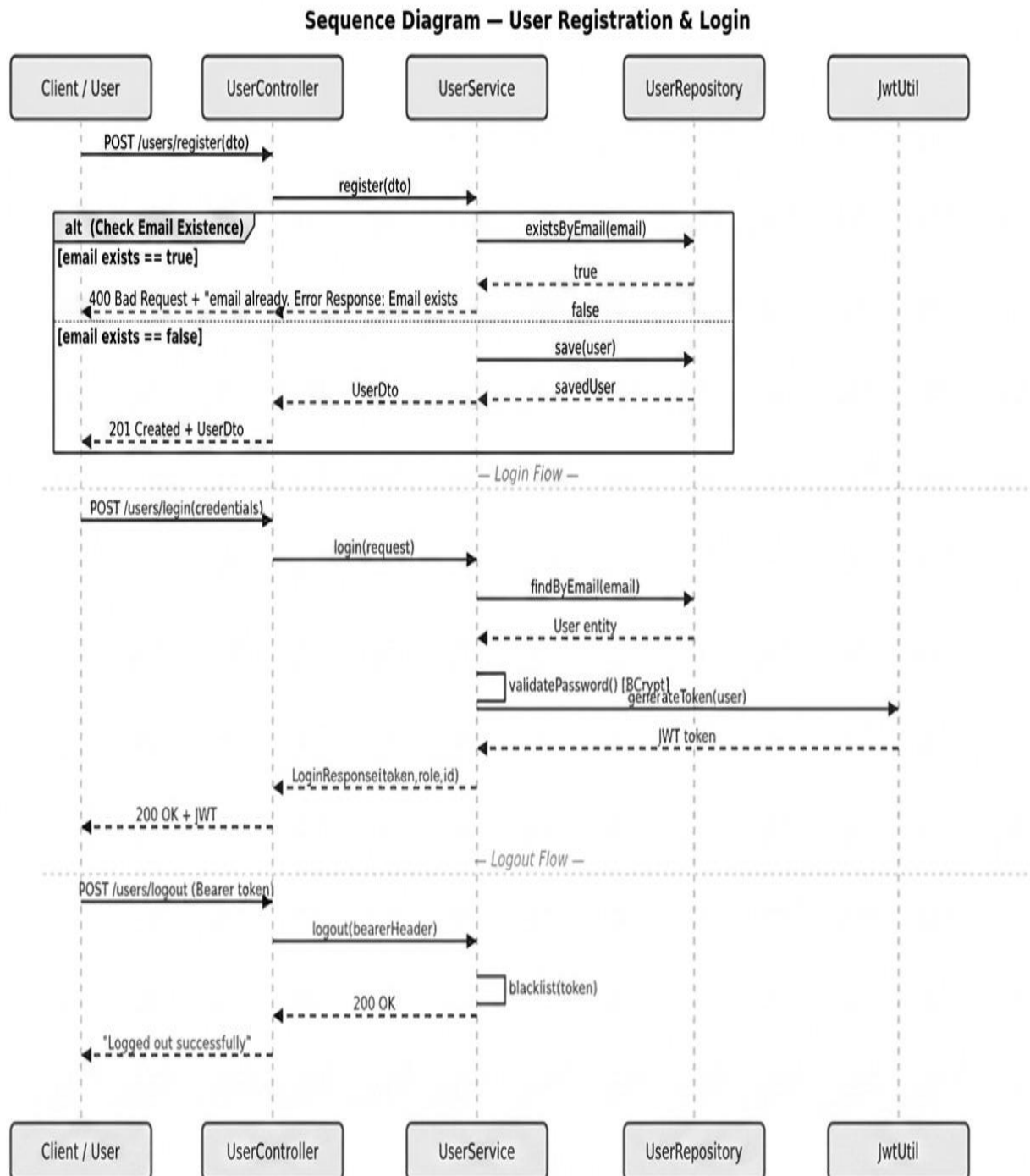
Activity Diagram:



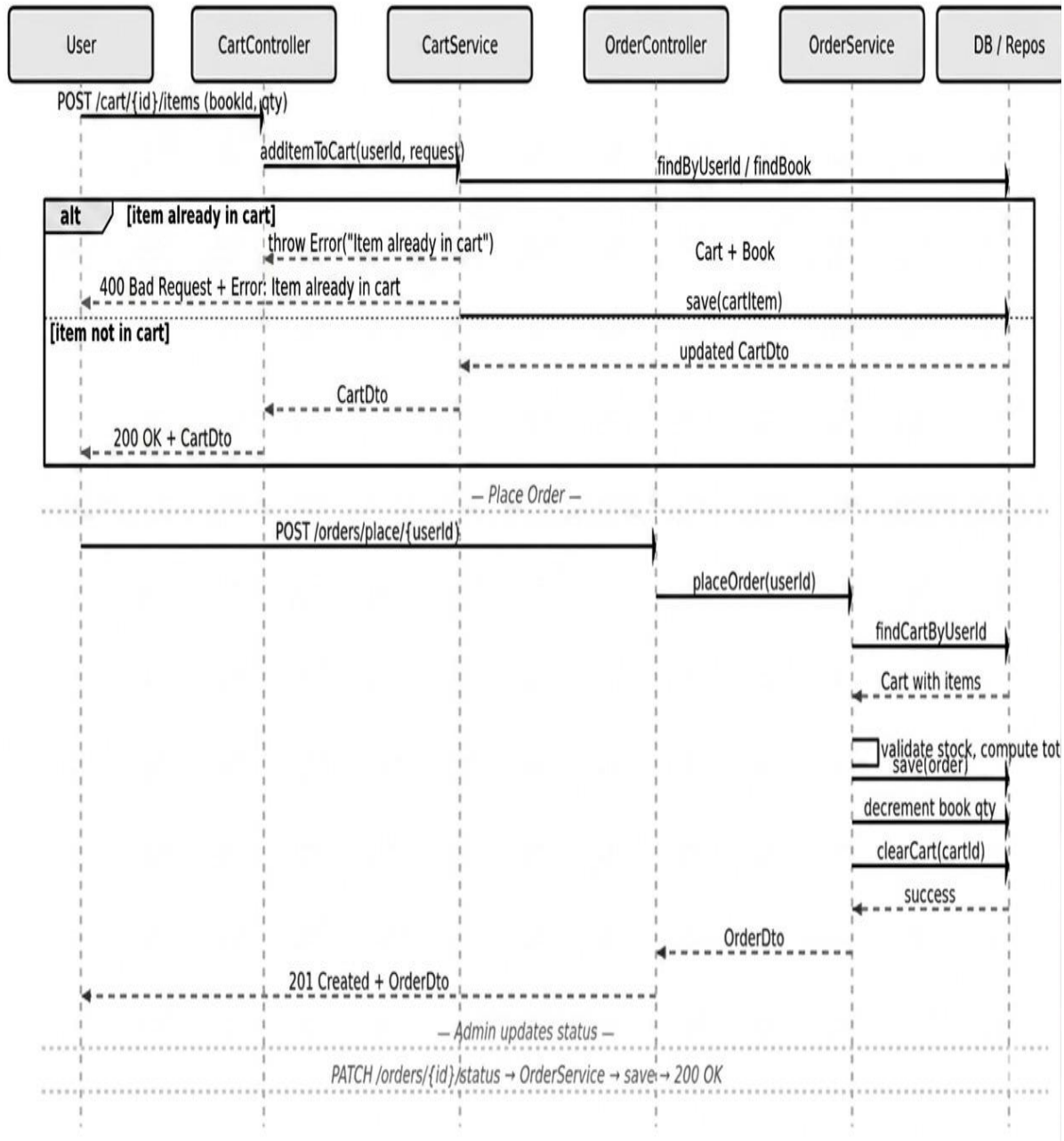
Add Book — Activity Diagram (Admin)



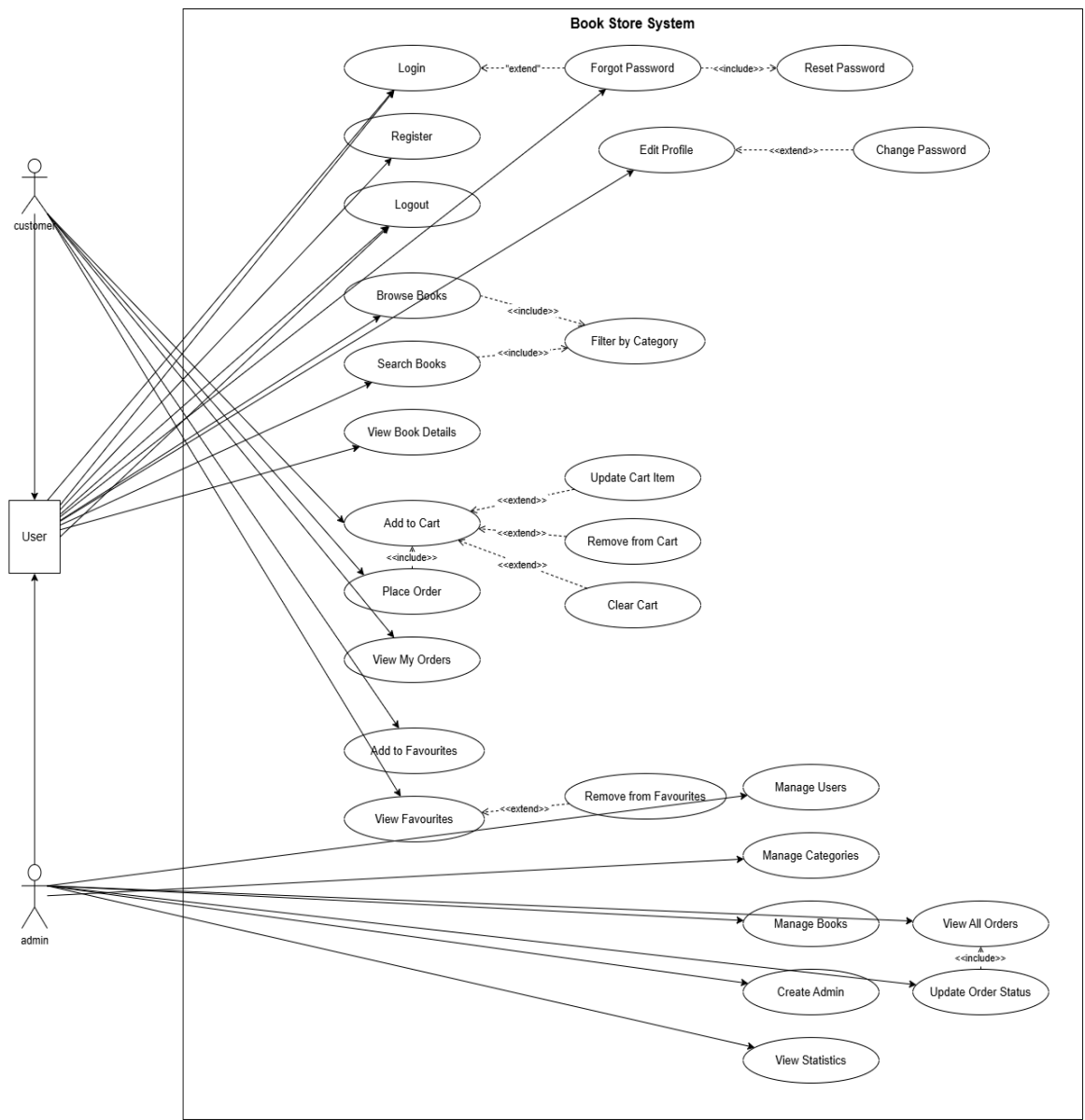
Sequence Diagram:



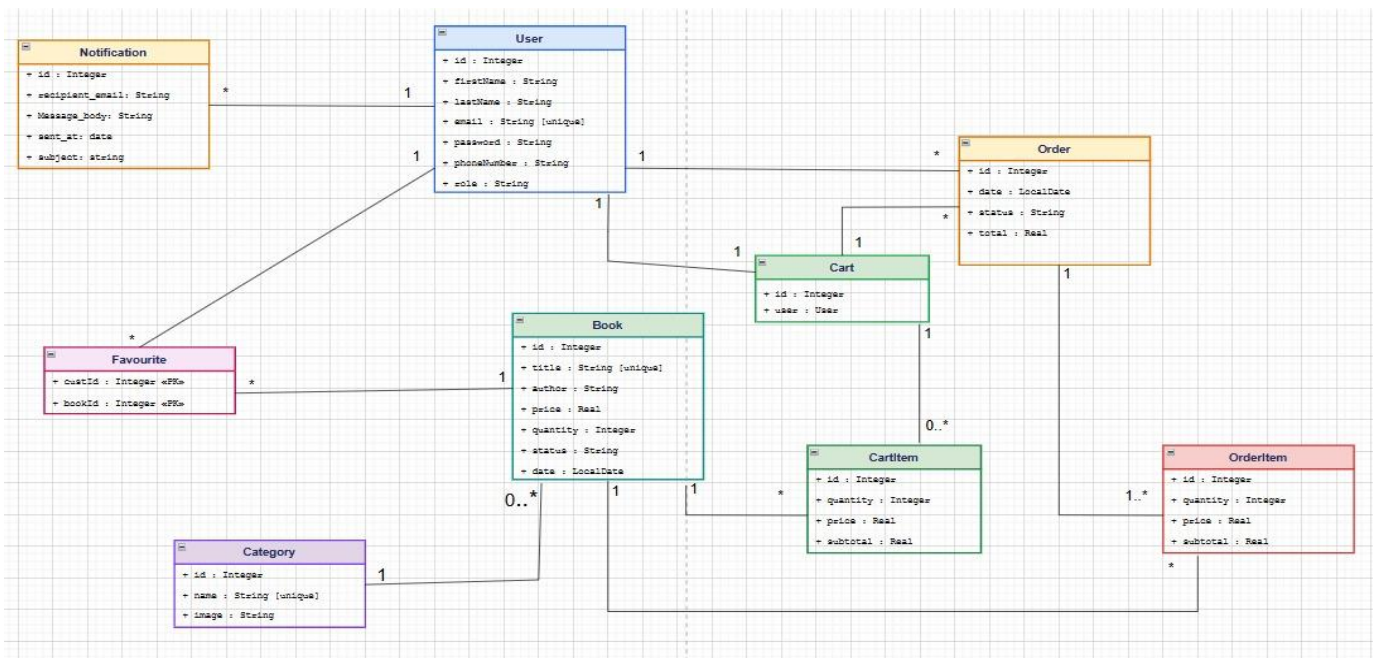
Sequence Diagram – Place Order



UseCase Diagram:



Class Diagram:



User:

«Context User» inv UniqueEmail: User.allInstances()->isUnique (u | u.email)

«Context User» inv EmailNotEmpty: self.email <>

«Context User» inv ValidRole: self.role = 'user' or self.role = 'admin'

Book:

«Context Book» inv NonNegQty: self.quantity >= 0

«Context Book» inv PositivePrice: self.price > 0

«Context Book» inv TitleNotEmpty: self.title <>

«Context Book» inv StatusSync: self.quantity <= 0 implies self.status = 'out of stock' and self.quantity > 0 implies self.status = 'in stock'

favorites:

«Context Favourite » inv UniqueFav: self.favourites->isUnique(f | f.bookId)

«Context Favourite» inv CompositeKey: Favourite.allInstances() ->isUnique(f | f.custId.toString() + '_' + f.bookId.toString())

«Context Favourite » inv OneOwner: self.user <> null

Cart:

«Context Cart» inv InStockOnly: self.cartItems->forAll (ci | ci.book.quantity > 0)

«Context Cart» inv OneOwner: self.user <> null

CartItem:

«Context CartItem» inv SubtotalCalc: self.subtotal = self.price * self.quantity

«Context CartItem » inv OneOwner: self.user <> null

Order:

«Context Order» inv PositiveTotal: self.total > 0

«Context Order»inv ValidStatus:self.status = 'pending'or self.status = 'delivered'

«Context Order»inv HasItems:self.orderItems->notEmpty()

OrderItem:

«Context OrderItem::checkout()»post SubtotalOk:self.subtotal = self.price * self.quantity

«Context OrderItem»inv PositiveQty:self.quantity > 0

«Context OrderItem»inv PositiveQty:self.price > 0

Category:

«Context Category»inv UniqueCategory: Category.allInstances()->isUnique(c | c.name)